

BSDA2001P: Introduction to DL and GenAI Project

Classification on Messy Mashups

Kaushal Vaid
24f2000681

Abstract

This report shows the approach and experiments done on the messy audio genre classification on the noisy, chaotic mashups (unknown recipe of the test data). The approach transforms raw audio into multi-channel Mel-Spectrograms and simulates test-time conditions through aggressive data augmentation, including per-stem tempo stretching, ESC-50 noise injection, and MixUp. The methodology evaluates a progression of three deep learning Convolutional Architectures (CNNs) – from very light weight custom-built CNN to a more deep Custom CNN, to a ResNet18 finetune and finally ending with EfficientNet-B3 finetuning and achieving a leaderboard score of **0.80933**.

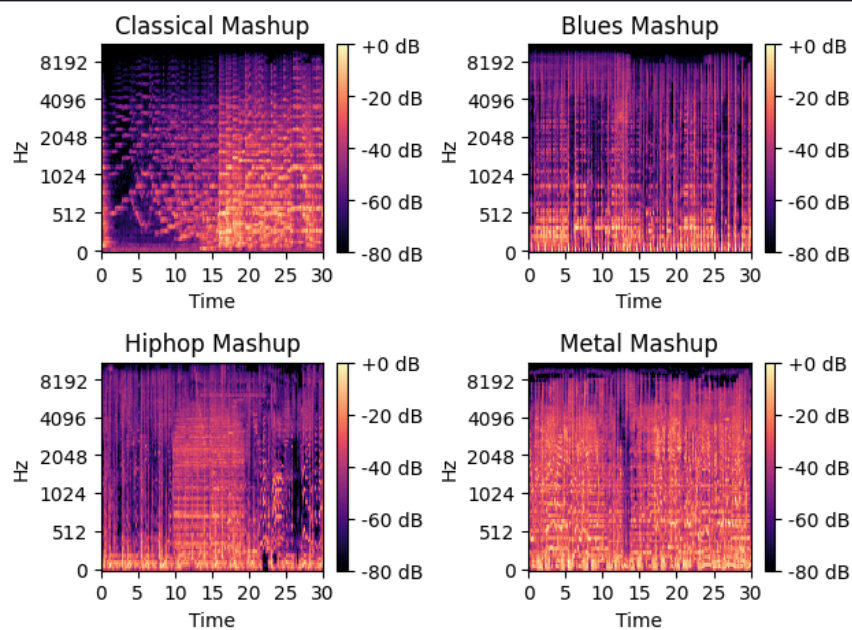
1. Introduction

This project addresses the challenge of robust music genre classification under highly degraded chaotic conditions, specifically focusing on the severe domain shift between clean training data and noise, real-world mixes of test data. The evaluation data has complex mashups heavily corrupted by tempo synchronization artifacts and additive environmental noise like sirens, vacuums, baby crying, dog barking etc (the “noises” from ESC-50).

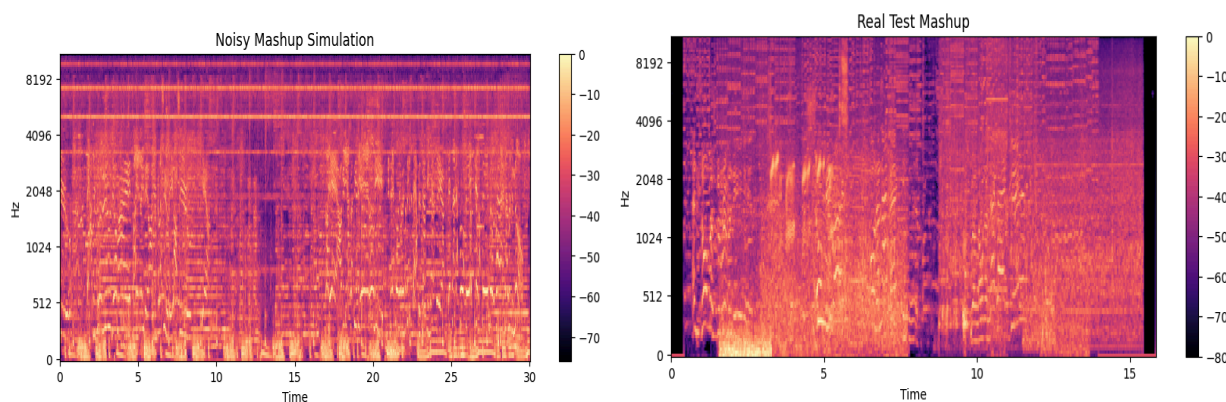
To mimic the test data, first I am framing the problem as an image classification task using multi-channel Mel-Spectrograms and relying heavily on dynamic data augmentations such as **cross-stem mixing, time stretching, MixUp, and aggressive noise injection**.

2. Exploratory Data Analysis (EDA) & Preprocessing Evolution

The initial exploratory data analysis was critical in diagnosing why baseline (ResNet50 and basic CustomCNN) failed to generalise. By plotting the 1D waveforms and converting them into 2D Mel-Spectrograms, distinct visual footprints emerged for different genres: Classical music presented as clean, horizontal bands, while HipHop showed bold, evenly spaced vertical lines, and metal appeared as dense, high intensity block across all frequencies.



Most critical insight came from comparing a clean training mix to a simulated test track. The test simulation revealed dense visual fog across the spectrogram caused by ESC-50 environmental noise.



Preprocessing V1 (And Why it failed)

Initially I simply summed the four clean stems **50% of the times** (drums, vocals, bass, other) into mixed track and added a single random noise clip at a high signal to noise ratio (SNR of 5 to 20dB). It extracted a standard 1-channel Mel-Spectrogram.

It failed: Because it was too gentle and just 50% of times it was having stem mixing, but in test, all data was cross-stem mixed. Therefore, the scratch CNN was not even exceeding **0.1 f1_macro** on val set, and ResNet gave **0.49** on leaderboard.

Test data had tempo-shifted stems and heavily layered, dominant noise. Therefore my model never learned these, resulting in fail.

Preprocessing V2 (The Aggressive and Robust Pipeline)

To fix it, I completely started from scratch and made the following changes:

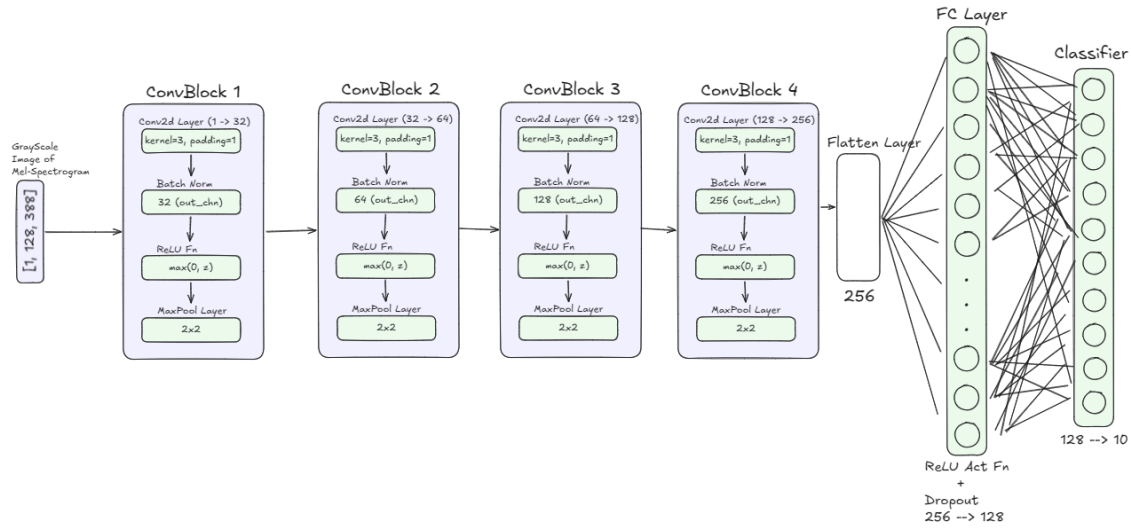
- a. **100% Cross stem mixing** - In this pipeline i made sure each training data is crafted from randomly cross-stem mixing.
- b. **Per-Stem Tempo Augmentation** - Instead of mixing stems at their native speed, individual stems were randomly time-stretched by up to 10% before mixing. This forced the model to learn the core acoustic texture (timbre) of the instruments rather than memorizing a specific Beats-Per-Minute (BPM) spacing.
- c. **Noise Injection** - The noise intensity was cranked up to an SNR of 0–15 dB (where 0 dB means the background noise is just as loud as the music), and occasionally, multiple noise clips were layered simultaneously. This brutal training environment acted as a powerful regularizer.
- d. **Dynamic Feature Extraction (Mel + Delta + Delta-Delta)** - Instead of a static 1-channel image, the pipeline calculated the first derivative (velocity of sound change) and second derivative (acceleration of sound change) over time. Stacking these created a natively 3-channel input that explicitly highlighted rhythmic changes, significantly boosting performance on beat-heavy genres.
- e. **MixUp Augmentation** - Blends training spectrograms and their labels to create smooth decision boundaries, preventing the model from overfitting to specific clean timbres and improving its ability to classify multi-instrumental mashups.
- f. **OneCycleLR with Warmup** - Utilizes a dynamic learning rate that starts low to safely stabilize initial weights, peaks rapidly to escape local minima, and smoothly decays to ensure robust convergence despite the chaotic noise.
- g. **5-Crop Test-Time Augmentation (TTA)** - Slices each test track into five overlapping temporal windows and averages their predictions during inference, drastically reducing prediction variance caused by localized bursts of extreme noise.

3. Architectural Comparisons

3.1 ScratchCNN (Version 1)

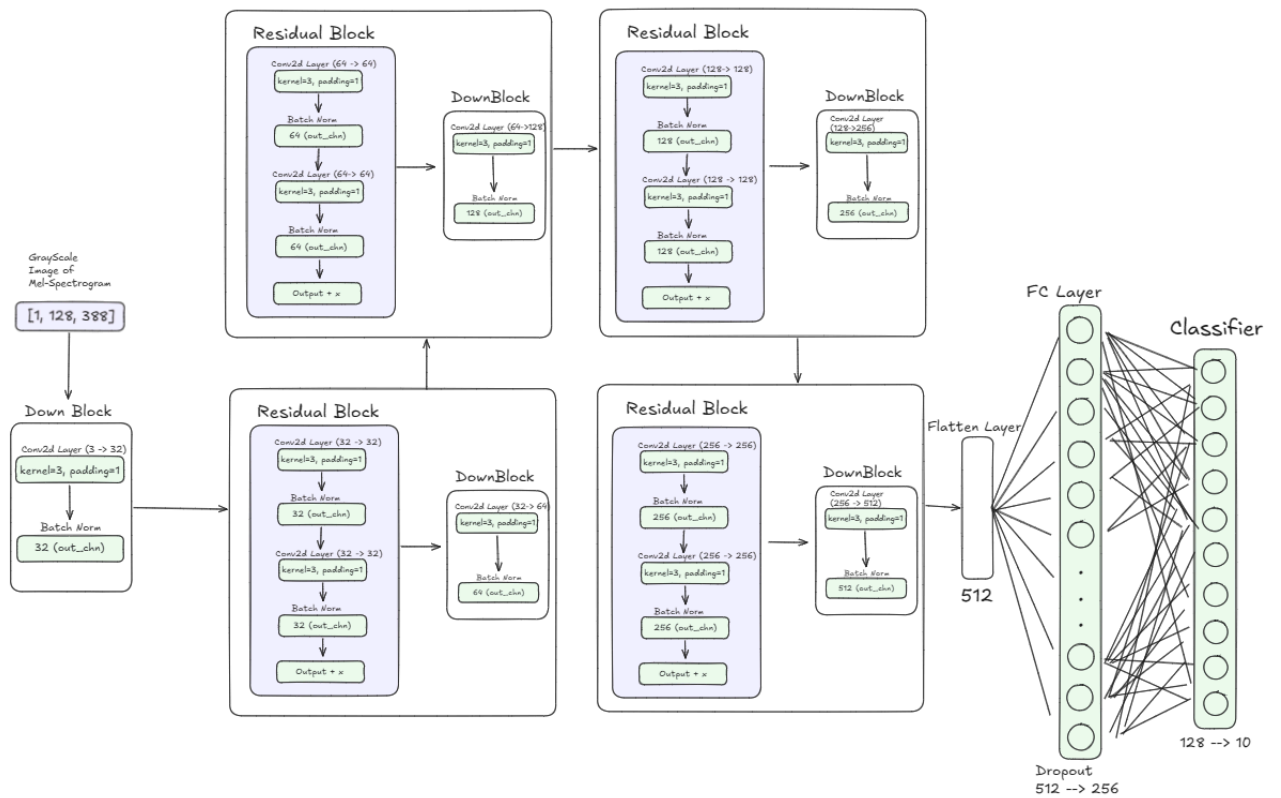
A scratch 4 block CNN using grayscale (1- channel) input followed by a Adaptive Average Pooling layer followed by Fully-Connected Layer with dropout.

Each conv block consist of 1 Convolution layer with kernel 3 and padding 1 followed by batch norm layer and ReLU activation function and a MaxPooling Layer of 2x2.



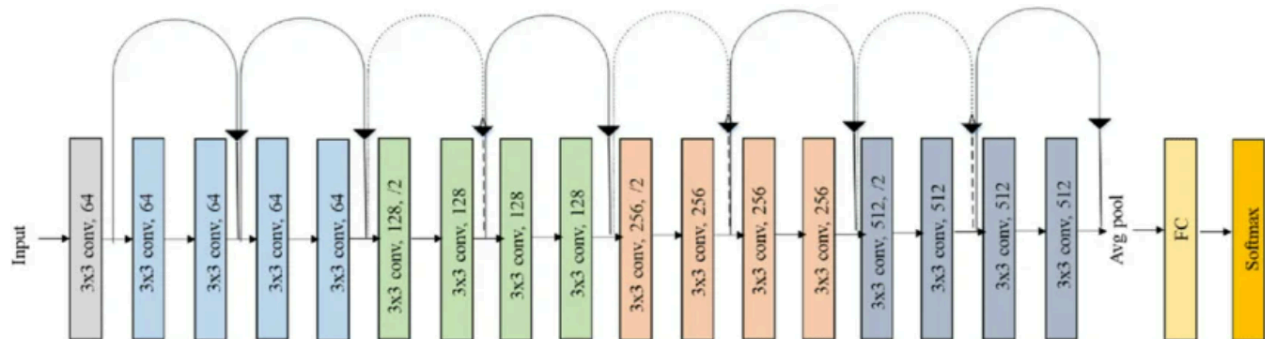
3.2 CustomCNN (Version 2)

Architecture changes here - Residual Block (Inspired by ResNet) + Strided Convolutional Block + Down Sampling + Spatial Attention and finally followed by Global Average Pooling (GAP) layer and then the Fully Connected layer with Classifier head.



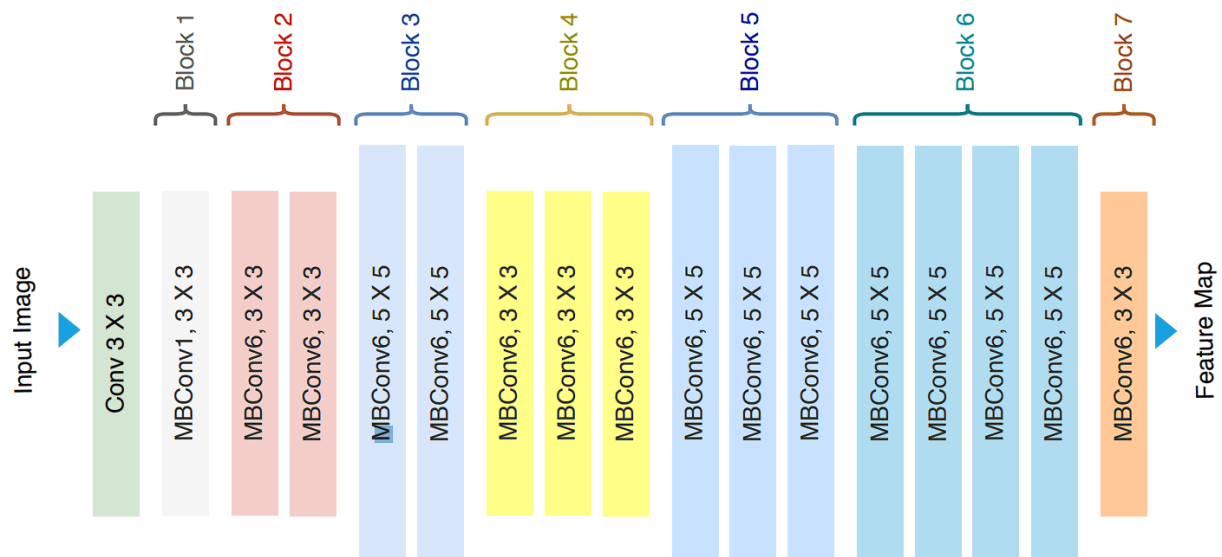
3.3 ResNet18 (Baseline)

ResNet utilized skip connections to mitigate vanishing gradients Problem. It proved to be reliable workhorse but a very unstable convergence (when checked the logs on wandb).



3.4 EfficientNet-B3 (Best Performing)

EfficientNet-B3 uses compound scaling to balance depth, width, and resolution. Why it was superior: The inclusion of **Squeeze-and-Excitation (SE)** blocks allowed the model to perform channel-wise attention, effectively “turning out” noise-heavy frequency bands while amplifying genre-specific ones.



4. Results and Performance

Monitoring and Logging was conducted through Weights and Biases (WandB) to track real-time convergence.

The table below summarizes the final Kaggle Leaderboard scores.

Model Name	Architecture	Version	Leaderboard Score
CustomCNN	4 Block CNN	Version 1	Didn't submit (val f1 - 0.1*)
CustomCNN	4 Residual Block CNN	Version 2	0.70105
resnet50-base	ResNet50	Baseline	0.49205
resnet18-mashup	ResNet18	Version 1	0.80869
EfficientNet-B3	EfficientNet	Version 1	0.80933

5. Conclusion

The iterative process revealed that the training data had to be as chaotic and degraded as the hidden test set. By overhauling the preprocessing pipeline to include 100% cross-stem mixing, per-stem tempo stretching, and aggressive 0-15 SNR noise injection, the domain gap was effectively bridged. Furthermore, upgrading the input representation to a 3-channel dynamic format gave the models the temporal context needed to distinguish rhythmic genres from harmonic ones. The jump from struggling on 0.1 to get a baseline of 0.49 to 0.80933 f1 macro underscores the importance of aligning dataset generation with the specific receptive field of the model architecture.

6. References

For understanding audio preprocessing:

- DL for audio classification (Seth Adams)
- Audio Signal Processing (Valerio Velardo)

To understand EfficientNet - [Complete Arch Details](#) [GeeksForGeeks](#)

Courses: DLGenAI Lectures, CampusX (100-days of DL)